Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 2. Iterative and Numeric Algorithms

Session 1

by:
`Er. Bikal Adhikari
(*B.E., M.Sc. Engg., M.Sc. ISE*)
Assistant Professor, Dept. of Electronics & Computer Engg.
Pulchowk Campus, IOE, TU

CHAPTER OUTLINE

Chapter 2. Iterative and Numeric Algorithms

8 hours

2.1 Algorithm for GCD and Fibonacci number
2.2 Sequential search
2.3 Review of bubble sort, selection sort, and insertion sort algorithms
2.4 Number theoretic notations
2.5 Euclid's and Extended Euclid's algorithms
2.6 Solving modular linear equations using Chinese remainder theorem
2.7 Fermat's theorem
2.8 Miller-Rabin randomized primility test and algorithm

ALGORITHM FOR GCD

Input: Two numbers a and b
Output: GCD of a and b
Algorithm:
Assumption a>b>=0

```
Algorithm GCD(a,b)
{
        while(b!=0)
        {
                d=a/b;
                temp=b;
                b=a-b*d;
                a=temp;
        }
        return a;
}
```

Efficiency:

- **Time Complexity:**

$$O(\log(min(a, b))).$$

- **Space Complexity:**

The only allocated spaces are for variables so space complexity is constant i.e. $O(1)$.

ALGORITHM FOR FIBONACCI NUMBERS

Input: *n*

Output: n^{th} Fibonacci number

Algorithm:

Assume ***a*** as first(previous) and ***b*** as second (current) numbers

```
Algorithm fibonacci(n)
{
if(n==0) return 0;
if(n==1) return 1;
a=0;b=1;
for(i=2;i<=n;i++)
{
    f=a+b;
    a=b;
    b=f;
}
return f;
}
```

Correctness:

The algorithm adds previous and current values i.e. *a* and *b*, and produces n^{th} fibonacci number(n>0) after n-1 iterations. So at each step the generated value will be correct. The algorithm terminates after n-1 iterations.

Efficiency:

- **Time Complexity:**

The algorithm above iterates upto n-1 times, so time complexity is $O(n)$.

- **Space Complexity:**

The space complexity is constant. i.e. $O(1)$.

SEQUENTIAL SEARCH ALGORITHM

Input: Search ***key*** and a list of *n* numbers
Output: Index of the searched item if found.
Algorithm:

```
Algorithm SequentialSearch(A, n, key)
{
        for(i=0;i<n;i++)
        {
                if(A[i]==key)
                        return i;
        }
        return -1; //  indicates unsuccessful search
}
```

Time Complexity = $O(n)$

SORTING ALGORITHMS

→ Sorting algorithms are used to sort data items in either ascending order or descending order.

→ In this section we will study different iterative sorting algorithms such as bubble sort, selection sort and insertion sort.

BUBBLE SORT ALGORITHM

Input: An array of size *n*

Output: Sorted array

Algorithm:

```
Algorithm BubbleSort(A, n)
{
i,j;
for(i=0;i<n-1;i++)
{
     for(j=0;j<n-i-1;j++)
     {
          if(A[j]>A[j+1])
          {
          swap(A[j],A[j+1];
          }
     }
}
}
```

No. of passes = *n*-1

Total no. of comparisons

= 1+2+… …+(n-1)

=n(n-1)/2 = $O(n^2)$

Total no. of swaps

= 1+2+… …+(n-1)

=n(n-1)/2 = $O(n^2)$

Time Complexity:

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on.

$T(n) = (n-1) + (n-2) + \ldots \ldots + 2 + 1$

$T(n)=O(n^2)$

Space Complexity:

Since no extra space besides 3 variables is needed for sorting.

$S(n)=O(n)$

Suppose a given array is:

$$A = \{24, 63, -5, 8, 1\}$$

The use of bubble sort to sort elements in A is illustrated below:

$n = 5$

	0	1	2	3	4
	24	63	-5	8	1
Pass 1	24	63	-5	8	1
	24	63	-5	8	1
	24	-5	63	8	1
	24	-5	8	63	1
	24	-5	8	1	**63**
Pass 2	24	-5	8	1	**63**
	-5	24	8	1	**63**
	-5	8	24	1	**63**
	-5	8	1	**24**	**63**
Pass 3	-5	8	1	**24**	**63**
	-5	8	1	**24**	**63**
	-5	1	**8**	**24**	**63**
Pass 4	**-5**	**1**	**8**	**24**	**63**

Result:

A={-5,1,8,24,63}

Suppose a given array is:

$$A = \{8, 5, 7, 3, 2\}$$

The use of bubble sort to sort elements in A is illustrated below:

$n = 5$

	0	1	2	3	4
	8	5	7	3	2
Pass 1	8	5	7	3	2
	5	8	7	3	2
	5	7	8	3	2
	5	7	3	8	2
	5	7	3	2	**8**
Pass 2	5	7	3	2	**8**
	5	7	3	2	**8**
	5	3	7	2	**8**
	5	3	2	**7**	**8**
Pass 3	5	3	2	**7**	**8**
	3	5	2	**7**	**8**
	3	2	**5**	**7**	**8**
Pass 4	3	2	**5**	**7**	**8**
	2	**3**	**5**	**7**	**8**

Result:

A={2, 3, 5, 7, 8}

SELECTION SORT ALGORITHM

Input: An array of size *n*

Output: Sorted array

Algorithm:

```
Algorithm SelectionSort(A, n)
{
for(i=0;i<n;i++)
{
Least=A[i];p=i;
    for(j=i+1;j<n;j++)
    {
        if(A[j]<least)
        {
          least=A[j];p=j;
        }
    } //inner loop closed
swap(A[i],A[p]);
}// outer loop closed
}
```

Time Complexity:

Inner loop executes for (n-1) times when i=0, (n-2) times when i=1 and so on.

$T(n) = (n-1) + (n-2) + \ldots \ldots + 2 + 1$

$T(n)=O(n^2)$

There is no best case linear time complexity for this algorithm, but number of swap operations is reduced considerably.

Space Complexity:

Since no extra space besides 5 variables is needed for sorting.

$S(n)=O(n)$

Suppose a given array is: A = {24, 63, -5, 8, 1}

The use of selection sort to sort elements in A is illustrated below:

$i = 0$

0	1	p = 2	3	4
24	63	-5	8	1

Swap A_i and A_p i.e. A_0 and A_2

$i = 1$

0	1	2	3	p = 4
-5	63	24	8	1

Swap A_i and A_p i.e. A_1 and A_4

$i = 2$

0	1	2	p = 3	4
-5	1	24	8	63

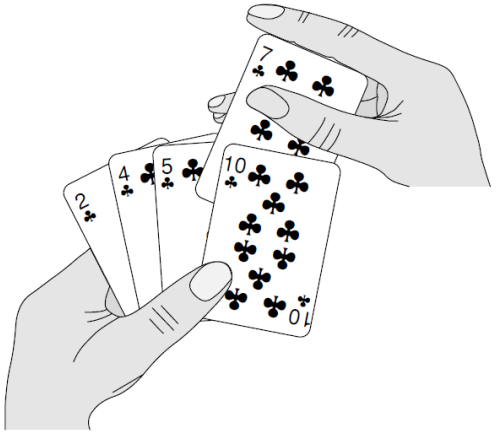
Swap A_i and A_p i.e. A_2 and A_3

$i = 3$

0	1	2	p = 3	4
-5	1	8	24	63

Swap A_i and A_p i.e. A_3 and A_3

INSERTION SORT ALGORITHM



Input: An array of size *n*

Output: Sorted array

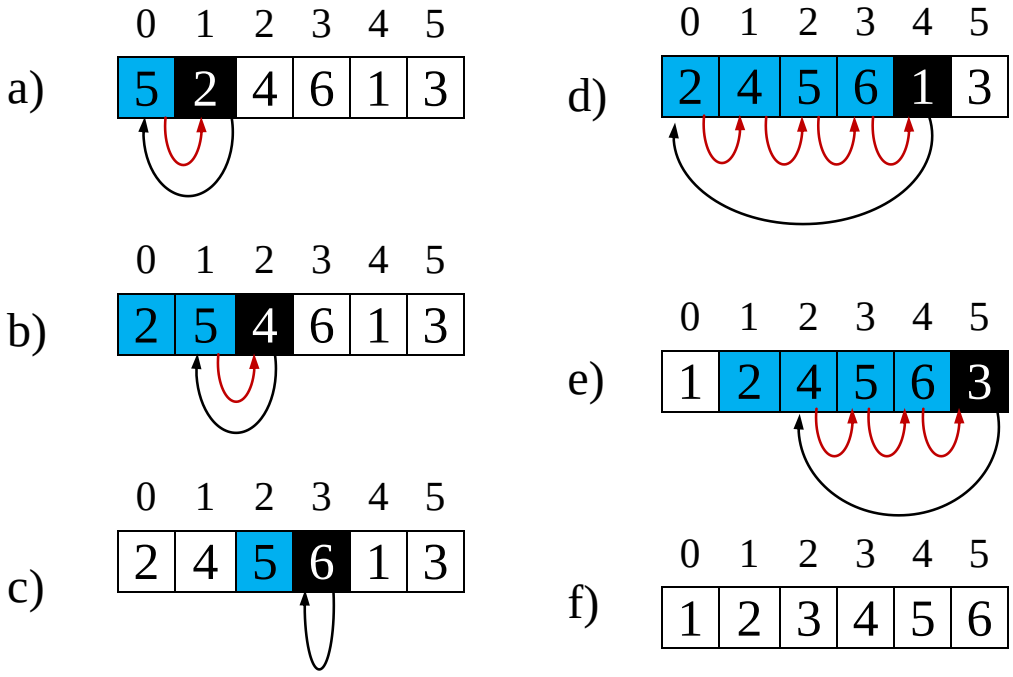
Algorithm:

```
Algorithm InsertionSort(A,n)
{
    for (i = 1; i < n; i++)
    {
        key = A[i];
        j = i - 1;
        while (j >= 0 && A[j] > key)
        {
            A[j + 1] = A[j];
            j = j - 1;
        }
        A[j + 1] = key;
    }
}
```

Suppose a given array is:

$$A = \{5, 2, 4, 6, 1, 3\}$$

The use of insertion sort to sort elements in A is illustrated below:



Time Complexity:

Worst Case Analysis: Array elements are in the reverse sorted order.

Inner loop executes for 1 time when i = 1, 2 times when i = 2 and n-1 times when $i = n$-1.

$T(n) = 1 + 2 + \ldots \ldots \ldots + (n\text{-}2) + (n\text{-}1)$

$T(n) = O(n^2)$

Best Case Analysis: Array elements are already sorted.

Inner loop executes 1 time when i = 1, 1 times when i = 2 ….and 1 time when $i = n$-1.

$T(n)=1+1+1\ldots\ldots\ldots\ldots..+1+1 = O(n)$

Space Complexity:

Since no extra space besides 5 variables is needed for sorting.

$S(n) = O(n)$

Thank You!!!

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms

Chapter 2. Iterative and Numeric Algorithms

Session 2

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 2. Iterative and Numeric Algorithms

8 hours

2.1 Algorithm for GCD and Fibonacci number
2.2 Sequential search
2.3 Review of bubble sort, selection sort, and insertion sort algorithms
2.4 Number theoretic notations
2.5 Euclid's and Extended Euclid's algorithms
2.6 Solving modular linear equations using Chinese remainder theorem
2.7 Fermat's theorem
2.8 Miller-Rabin randomized primility test and algorithm

Number Theoretic Notations

- This section provides a brief review of notions from elementary number theory concerning the set $\mathbb{Z}$ = {….., -2, -1, 0, 1, 2, …..} of integers and the set N = {0, 1, 2, 3, 4, ….. } of natural numbers.

Integers

- An integer is the number zero (0), a positive natural number (1, 2, 3, etc.) or a negative integer with a minus sign (−1, −2, −3, etc.).
- The negative numbers are the additive inverses of the corresponding positive numbers.
- In the language of mathematics, the set of integers is often denoted by the symbol $\mathbb{Z}$.

Integers and Division

Divisibility of Integers:

We say that an integer a divides b or b is divisible by a if there exists an integer k such that

$$b = ak.$$

If it is the case, we also write $a|b$. We also say that a is a divisor (or a factor) of b.

On the other hand, if a does not divide b, we write $a \nmid b$

Remainder:

The remainder of the division of b with a is an integer r when there exists an integer q such that

$$b = qa + r$$

Where,

$$0 \leq r < a$$

We refer to q as the quotient of the division.

We use operator mod to denote an operation for finding the remainder of a division. i.e. a mod b is the remainder of dividing a with b.

Prime and Composite Numbers

- An integer $p>1$ is a prime number if its divisors are only p, $-p$, 1 and -1.
- If an integer $n>1$ is not a prime, it is called a composite.

Note: **1 is neither a prime number nor a composite number.**

Greatest Common Divisors (GCD)

- For integers *x* and *y*, the greatest common divisor(or GCD) of *x* and *y* is the largest integer *g* such that *g*|*x* and *g*|*y*. We refer to it as *gcd*(*x*,*y*).
- For example: gcd (24,30) = 6, gcd(5,7)= 1, gcd(0,9)=9
- If a and b are not both 0, then gcd(a,b) is an integer between 1 and min(|a|, |b|). We define gcd(a,b) to be 0; this definition is necessary to make standard properties of the gcd() function universally valid.
- The following are elementary properties of the gcd function:

$$\gcd(a,b) = \gcd(b,a)$$

$$\gcd(a,b) = \gcd(-a,b)$$

$$\gcd(a,b) = \gcd(|a|,|b|)$$

$$\gcd(a,0) = |a|$$

$$\gcd(a,ka) = |a| \text{ for any } k \in Z$$

- Corollary 1:

For any integer a and b, if $d|a$ and $d|b$ then $d|gcd(a,b)$.

- Corollary 2:

For all integers a and b and any non-negative integer n,

$$gcd(a.n,b.n) = n.gcd(a,b)$$

- Corollary 3:

For all positive integers n, a and b, if $n|ab$ and $gcd(a,n) = 1$, then $n|b$.

Relatively Prime Integers

- Two integers a and b are said to be relatively prime integers if their only common divisor is 1, that is if $gcd(a,b) = 1$.
- For example: 8 and 15 are relatively prime , since the divisors of 8 are 1, 2, 4 and 8 while the divisors of 15 are 1, 3, 5, and 15.
- Following are the theorems on relatively prime integers:
  - **Theorem 1:**

  For any integers a, b, and p, if $gcd(a,p) = 1$ and $gcd(b,p) = 1$, then

  $$gcd(ab,p) = 1.$$

  - **Theorem 2:**

  For all primes p and all integers a, b if $p|a.b$ then $p|a$ or $p|b$ or both.

Euclid's Algorithm

- For integers x and y, the greatest common divisor(or GCD) of x and y is the largest integer g such that $g|x$ and $g|y$. We refer to it as $gcd(x,y)$.
- The Euclid's Algorithm to find GCD is:

```
Algorithm Euclid(x,y):
    if x mod y ==0:
        return y
    else:
        return Euclid(y, x mod y)
```

- For example: Euclid(18,12) = Euclid(12,6) = 6

Useful Fact

- For any integer x and y, consider the term

  $a.x + b.y$

  for some integer *a* and *b*.
- When the term is non-zero, it must be divisible by $gcd(x,y)$, so it has to be at least $gcd(x,y)$.
- It turns out that you can actually attain that value. i.e. there exist a pair of integer *a* and *b* such that

$$a.x+b.y = gcd(x,y)$$

Extended Euclidean Algorithm

```
Algorithm Euclid(x,y):
if x mod y ==0:
   return y, 0, 1
else:
   g,a′,b′=Euclid(y,x mod y)
   a=b′
   b=a′-b′*floor(x/y)
   return g,a,b
```

Time Complexity Analysis

The Euclidean algorithm computes the greatest common divisor (GCD) of two integers by repeatedly applying the rule

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

At each step, the pair (a, b) is replaced with (b, a mod b), and since (a mod b) < b, the second argument strictly decreases, ensuring termination.

The slowest decrease occurs when the inputs are consecutive Fibonacci numbers. For example,

$$\gcd(F_{n+1}, F_n) \rightarrow \gcd(F_n, F_{n-1}) \rightarrow \ldots \rightarrow \gcd(F_2, F_1)$$

This sequence requires exactly n recursive calls. Fibonacci numbers represent the worst case because they minimize the reduction at each step, making the algorithm progress most slowly.

Fibonacci numbers grow exponentially according to the exact Binet's formula, $F_n = (\varphi_1^n - \varphi_2^n) / \sqrt{5}$, where $\varphi_1 = (1 + \sqrt{5}) / 2 \approx 1.618$ and $\varphi_2 = (1 - \sqrt{5}) / 2 \approx -0.618$

Here the second term $\varphi_2^n / \sqrt{5}$ decays rapidly because $|\varphi_2| < 1$, so for large n it becomes negligible. This allows us to approximate,

$$F_n \approx \varphi_1^n / \sqrt{5}$$

If the smaller input is b, then $b \approx F_n$, which implies

$$n \approx \log_{\varphi_1}(b)$$

Therefore, the number of recursive calls is bounded by

$$T(a, b) \leq \log_{\varphi_1}(\min(a, b))$$

Since $\log_{\varphi_1}(b) = \log(b) / \log(\varphi_1)$, this is asymptotically

$$T(a, b) = O(\log(\min(a, b)))$$

Thus, even in the worst case, the recursion depth grows only logarithmically with the input magnitude.

Example: ***Find the gcd of* 161 *and* 28 *using extended Euclidean algorithm. Also find the values of a and b such that***

$$161a+28b = gcd(161,28).$$

***Solution*:**

We are required to find the gcd of 161 mod 28 using EEA.

Initially, we put $x = 161$, $y = 28$, $a_1 = 1$, $a_2 = 0$, and $b_1 = 0$, $b_2 = 1$.

q	x	y	r	a_1	a_2	a	b_1	b_2	b

Example: ***Find the gcd of* 161 *and* 28 *using extended Euclidean algorithm. Also find the values of a and b such that***

$$161a+28b = gcd(161,28).$$

***Solution*:**

We are required to find the gcd of 161 mod 28 using EEA.

Initially, we put $x = 161$, $y = 28$, $a_1 = 1$, $a_2 = 0$, and $b_1 = 0$, $b_2 = 1$.

q	x	y	r	a_1	a_2	a	b_1	b_2	b
	161	28		1	0		0	1	

Example: ***Find the gcd of* 161 *and* 28 *using extended Euclidean algorithm. Also find the values of a and b such that***

$$161a+28b = gcd(161,28).$$

***Solution*:**

We are required to find the gcd of 161 mod 28 using EEA.

Initially, we put $x = 161$, $y = 28$, $a_1 = 1$, $a_2 = 0$, and $b_1 = 0$, $b_2 = 1$.

The value of q is the quotient of x/y and r is the remainder obtained from x mod y.

q	x	y	r	a_1	a_2	a	b_1	b_2	b
	161	28		1	0		0	1	

Example: ***Find the gcd of* 161 *and* 28 *using extended Euclidean algorithm. Also find the values of a and b such that***

$$161a+28b = gcd(161,28).$$

***Solution*:**

We are required to find the gcd of 161 mod 28 using EEA.

Initially, we put $x = 161$, $y = 28$, $a_1 = 1$, $a_2 = 0$, and $b_1 = 0$, $b_2 = 1$.

The value of q is the quotient of x/y and r is the remainder obtained from x mod y.

q	x	y	r	a_1	a_2	a	b_1	b_2	b
5	161	28	21	1	0		0	1	

Example: ***Find the gcd of* 161 *and* 28 *using extended Euclidean algorithm. Also find the values of a and b such that***

$$161a+28b = gcd(161,28).$$

***Solution*:**

We are required to find the gcd of 161 mod 28 using EEA.

Initially, we put $x = 161$, $y = 28$, $a_1 = 1$, $a_2 = 0$, and $b_1 = 0$, $b_2 = 1$.

The value of q is the quotient of x/y and r is the remainder obtained from x mod y.

q	***x***	***y***	***r***	a_1	a_2	***a***	b_1	b_2	***b***
5	161	28	21	1	0	1	0	1	-5

The value of a and b are obtained by using formula:

$a = a_1 - q \times a_2$ and $b = b_1 - q \times b_2$

Left shift the values of (y,r), (a_2,a) and (b_2, b) to get the values of (x,y), (a_1,a_2) and (b_1, b_2) in the next row as indicated.

q	***x***	***y***	***r***	***a_1***	***a_2***	***a***	***b_1***	***b_2***	***b***
5	161	28	21	1	0	1	0	1	-5
	28	21		0	1		1	-5	

Left shift the values of (y,r), (a_2,a) and (b_2, b) to get the values of (x,y), (a_1,a_2) and (b_1, b_2) in the next row as indicated.
Repeat the calculation in the new row as done previously.

q	***x***	***y***	***r***	a_1	a_2	***a***	b_1	b_2	***b***
5	161	28	21	1	0	1	0	1	-5
1	28	21	7	0	1	-1	1	-5	6

Repeat the process for the next row,

q	x	y	r	a_1	a_2	a	b_1	b_2	b
5	161	28	21	1	0	1	0	1	-5
1	28	21	7	0	1	-1	1	-5	6
3	21	7	0	1	-1	4	-5	6	-23

Repeat the process for the next row,
Since the division is not possible, we stop here.
We mark other fields with ×.

q	x	y	r	a_1	a_2	a	b_1	b_2	b
5	161	28	21	1	0	1	0	1	-5
1	28	21	7	0	1	-1	1	-5	6
3	21	7	0	1	-1	4	-5	6	-23
×	7	0	×	-1	4	×	6	-23	×

GCD = value of x in the last row = 7

a = value of a_1 in the last row = -1

b = value of b_1 in the last row = 6

q	x	y	r	a_1	a_2	a	b_1	b_2	b
5	161	28	21	1	0	1	0	1	-5
1	28	21	7	0	1	-1	1	-5	6
3	21	7	0	1	-1	4	-5	6	-23
×	7	0	×	-1	4	×	6	-23	×

Linear Congruence

- Two integers a and b are congruent modulo m *iff* they have the same remainder when divided by m.
- For an integer $m>0$, if integers a and b are such that

$$a \bmod m = b \bmod m$$

We write,

$a \equiv b \pmod m$.

We also have that

$a \equiv b \pmod m \Leftrightarrow m \mid (a-b)$

Multiplicative Inverse modulo m

The multiplicative inverse modulo m of a, denoted by a^{-1}, is an integer such that

$$a.a^{-1} \equiv 1 \pmod{m}$$

Theorem: An integer *a* has a multiplicative inverse modulo *m iff* *gcd*(*a*,*m*) = 1

Example: Find the inverse of 5 mod 7

Solution:

$1 \equiv 8 \equiv 15 \pmod 7 = 5.3 \pmod 7$

$\therefore 5^{-1} \equiv 3 \pmod 7$

Example: Find the inverse of 5 mod 11

Solution:

$1 \equiv 12 \equiv 23 \equiv 34 \equiv 45 \pmod{11} = 5.9 \pmod{11}$

$\therefore 5^{-1} \equiv 9 \pmod{11}$

Example: Find the inverse of 6 mod 17

Solution:

$1 \equiv 18 \pmod{17} = 6.3 \pmod{17}$

$\therefore 6^{-1} \equiv 3 \pmod{17}$

Chinese Remainder Theorem

- The chinese Remainder Theorem is used to solve a set of different congruent equations with one variable but different modulii which are relatively prime.
- Let $m_1, m_2, \ldots, m_n$ be pair wise relatively prime positive integers greater than one and $a_1, a_2, \ldots a_n$ arbitrary integers. Then, the system:

$x \equiv a_1 \pmod{m_1}$.

$x \equiv a_2 \pmod{m_2}$.

.

.

.

$x \equiv a_n \pmod{m_n}$.

has a unique solution modulo $m = m_1 m_2 \ldots m_n$

(That is, there is a solution x with $0 \leq x < m$, and all other solutions are congruent modulo m to this solution.)

- The solution is of the form:

$$x = \left(M_1 X_1 a_1 + M_2 X_2 a_2 + \ldots\ldots + M_n X_n a_n\right) \bmod m$$

$$\text{where}, m = m_1 \times m_2 \times m_3 \times \ldots \times m_n \quad \text{and } M_i = \frac{m}{m_i}$$

$X_i = \text{Inverse of } M_i \text{ modulo } m_i$

Example:

There are certain things whose number is unknown. When divided by 3, the remainder is 2; when divided by 5, the remainder is 3; and when divided by 7, the remainder is 2. What will be the number of things?

This puzzle can be translated into the following question:

What are the solutions of the systems of congruences

$x \equiv 2 \pmod 3$,

$x \equiv 3 \pmod 5$,

$x \equiv 2 \pmod 7$?

Solution: $m_1 = 3$, $m_2 = 5$, $m_3 = 7$,$a_1 = 2$, $a_2 = 3$ and $a_3 = 2$

$m = m_1.m_2.m_3 = 3.5.7 = 105$

$$M_1 = \frac{m}{m_1} = \frac{3.5.7}{3} = 35$$

$$M_2 = \frac{m}{m_2} = \frac{3.5.7}{5} = 21$$

$$M_3 = \frac{m}{m_3} = \frac{3.5.7}{7} = 15$$

The solution is:

$$x = (M_1 X_1 a_1 + M_2 X_2 a_2 + M_3 X_3 a_3) \bmod m$$

$$x = (35.2X_1 + 21.3X_2 + 15.2X_3) \bmod 105$$

$$X_1 = \text{Inverseof } M_1 \bmod m_1 = \text{Inverseof } 35 \bmod 3$$

$$= 2[\because 1 \equiv 2.2 (\bmod 3) = 35.2 (\bmod 3)]$$

$$X_2 = \text{Inverseof } M_2 \bmod m_2 = \text{Inverseof } 21 \bmod 5$$

$$= 1[\because 21 \equiv 1 (\bmod 5)]$$

$$X_3 = \text{Inverseof } M_3 \bmod m_3 = \text{Inverseof } 15 \bmod 7$$

$$= 1[\because 15 \equiv 1 (\bmod 7)]$$

$$\therefore x = (35.2.2 + 21.3.1 + 15.2.1) \bmod 105$$

$$= (140 + 63 + 30) \bmod 105$$

$$= 233 \bmod 105$$

$$= 23$$

Pseudocode for Chinese Remainder Theorem

```
CHINESE-REMAINDER-THEOREM(a[1..n], m[1..n])
1. mtot ← 1
2. for i ← 1 to n
3.     mtot ← mtot * m[i]
4. sum ← 0
5. for i ← 1 to n
6.     M[i] ← mtot / m[i]
7.     X[i] ← MODINV(M[i], m[i])
8.     sum ← sum + a[i] * M[i] * X[i]
9. return sum mod mtot
```

```
MODINV(a, m)
1. (g, x, y) ← EXTENDED-EUCLID(a, m)
2. if g ≠ 1
3.     error "Inverse does not exist"
4. else
5.     return (x mod m + m) mod m
```

```
EXTENDED-EUCLID(x, y)
1. if y = 0
2.     return (x, 1, 0)
3. else
4.     (g, t1, t2) ← EXTENDED-EUCLID(y, x mod y)
5.     a ← t2
6.     b ← t1 - ⌊x / y⌋ * t2
7.     return (g, a, b)
```

Time Complexity of Chinese Remainder Theorem

Step	Operation	Typical Cost
1	Compute total modulus $m = m_1.m_2 \ldots .m_n$	$O(n)$ multiplications
2	Compute each $M_i = m/m_i$	$O(n)$ divisions
3	Compute modular inverses $X_i = M_i^{-1}\ m_i$	$O(\log m_i)$ per inverse using Extended Euclidean algorithm
4	Compute weighted sum x = $\sum M_i X_i a_i$	$O(n)$ multiplications and additions
5	Final reduction x mod m	$O(1)$

Overall Time Complexity: $O(n \cdot \log m_{max})$
Space Complexity: $O(n)$

Modular Exponentiation

- It is a type of exponentiation performed under modulus.
- It is written as:

$$a^b \bmod m \; or \; a^b \pmod{m}$$

- Examples: $2^{33} \bmod 30$, $3^{100} \bmod 29$

Pseudocode for Modular Exponentiation

```
MODULAR-EXPONENTIATION(a, b, m)
1   c = 0
2   d = 1
3   let ⟨b_k, b_{k-1}, …, b_0⟩ be the binary representation of b
4   for i = k downto 0
5       c = 2c
6       d = (d · d) mod m
7       if b_i == 1
8           c = c + 1
9           d = (d · a) mod m
10  return d
```

Time Complexity

→ The time complexity of the iterative implementation of MODULAR-EXPONENTIATION algorithm is

$$O(\log(min(a,b))) = O(\log b)$$

Examples

Example 1: Solve 23^3 mod 30.

Solution:

23^3 mod 30

= -7^3 mod 30 because 23 mod 30 can be 23 or 7.

= $(-7)^2$. -7 mod 30

= 49.(-7) mod 30

= -133 mod 30

= -13 mod 30

= 17 mod 30

∴ 23^3 mod 30 = 17

Example 2: Solve 31^{500} mod 30.

Solution:

31^{500} mod 30

$= 1^{500}$ mod 30

= 1 mod 30

=1

∴ 31^{500} mod 30 =1

Example 3: Solve 242^{329} mod 243.

Solution:

242^{329} mod 243

$= (-1)^{329}$ mod 243

= -1 mod 243

= 242 mod 243

= 242

∴ 242^{239} mod 243 = 242

Example 4: Solve 11^7 mod 13.

Solution:

$11^7 \bmod 13$

$= (-2)^7 \bmod 13$

$= (-2)^6.(-2) \bmod 13$

$= -128 \bmod 13$

$= -11 \bmod 13$

$= 2 \bmod 13$

$= 2$

$\therefore 11^7 \bmod 13 = 2$

Example 5: Solve 88^7 mod 187.

Solution:

$88^1 \bmod 187 = 88$

$88^2 \bmod 187 = 88^1.88^1 \bmod 187 = 88.88 \bmod 187$

$= 7744 \bmod 187 = 77$

$88^4 \bmod 187 = 88^2.88^2 \bmod 187 = 77.77 \bmod 187 =$

$= 5929 \bmod 187 = 132$

$88^7 \bmod 187 = 88^4.88^2.88^1 \bmod 187 = (132.77.88) \bmod 187$

$= 894432 \bmod 187 = 11$

$\therefore 88^7 \bmod 187 = 11$

Example 6: What is the last two digits of 29^5 ?

Solution:

We are required to find the value of $29^5 \bmod 100$.

Now,

$29^1 \bmod 100 = 29$

$29^2 \bmod 100 = 29^1 . 29^1 \bmod 100 = 841 \bmod 100 = 41$

$29^4 \bmod 100 = 29^2 . 29^2 \bmod 100 = 41.41 \bmod 100$

$= 1681 \bmod 100$

$= 81$

$29^5 \bmod 100 = 29^4 . 29^1 \bmod 100 = 81.29 \bmod 100$

$= 2349 \bmod 100$

$= 49$

$\therefore 29^5 \bmod 100 = 49$

Example 7: Solve 3^{100} mod 29 ?

Solution:

Answer: **3^{100} mod 29 = 20**

Example 8: Solve 23^{16} mod 30 ?

Solution:

Answer: **23^{16} mod 30 = 1**

Miler-Rabin Randomized Primility Test

- Known as Miller-Rabin primality test or Rabin-Miller primality test.
- It is a probabilistic primality test.
- It is similar to the Fermat primality test and the Sovovay-Strassen Primality test.
- Checks whether a specific property, which is known to hold for prime values, holds for the number under testing.

Algorithm

Algorithm

Step 1: Find $n-1=2^k \times m$

Step 2: Choose a such that $1 < a < n\text{-}1$

Step 3: Compute $b_0 = a^m \pmod{n}, \ldots, b_i = b_{i-1}^2 \pmod{n}$

$+1 \rightarrow$ Composite

$-1 \rightarrow$ Probably Prime

Step 4: If $b_i \neq -1$ and $b_i \neq 1$, then increment i and repeat step 3

until b_i becomes equal to -1 or +1

Time Complexity

Step	Operation	Typical Cost
Decompose (n-1)	Divide by 2 repeatedly	$O(\log n)$
Modular Exponentiation	Fast exponentiation	$O(\log^3 n)$
Repeat for t bases	Linear scaling	$O(t.\log^3 n)$

Example

Example 1: **Is 561 prime ?**

Solution:

Given $n = 561$

Step 1:

$n\text{-}1 = 2^k \times m$

or, $560 = 2^4 \times 35$

So, $k=4$ and $m = 35$

$$\frac{560}{2^1} = 280 \left| \frac{560}{2^2} = 140 \right| \frac{560}{2^3} = 70 \left| \frac{560}{2^4} = 35 \right| \frac{560}{2^5} = 17.5$$

Step 2:

Choosing $a = 2$ ($\because 1 < a < 560$)

Step 3:

$$b_0 = 2^{35} (\text{mod} 561) = 263$$

$$\text{Since}, b_0 \neq -1 \, or \, +1$$

$$b_1 = b_0^2 (\text{mod} 561)$$

$$= 263^2 (\text{mod} 561)$$

$$= 166$$

$$2^1 (\text{mod} 561) = 2,$$

$$2^2 (\text{mod} 561) = 4$$

$$2^4 (\text{mod} 561) = 16,$$

$$2^8 (\text{mod} 561) = 256 (\text{mod} 561) = 256$$

$$2^{16} (\text{mod} 561) = 65536 (\text{mod} 561) = 460$$

$$2^{32} (\text{mod} 561) = 211600 (\text{mod} 561) = 103$$

$$2^{35} (\text{mod} 561) = 103.4.2 (\text{mod} 561) = 824 (\text{mod} 561) = 263$$

$$\text{Since}, b_1 \neq -1 or +1$$

$$b_2 = b_1^2 (\text{mod} 561)$$

$$= 166^2 (\text{mod} 561)$$

$$= 67$$

$$\text{Since}, b_2 \neq -1 or +1$$

$$b_3 = b_2^2 (\text{mod} 561)$$

$$= 67^2 (\text{mod} 561)$$

$$= 1$$

Since, $b_3 = +1, n = 561$ is a composite number.

Example 2: **Is 53 prime ?**

Solution:

Given $n = 53$

Step 1:

$n\text{-}1 = 2^k \times m$

or, $52 = 2^2 \times 13$

So, $k=2$ and $m = 13$

$$\frac{52}{2^1} = 26 \left| \frac{52}{2^2} = 13 \right| \frac{52}{2^3} = 6.5$$

Step 2:

Choosing $a = 51$ ($\because 1 < a < 52$)

Step 3:

$$b_0 = 51^{13} (\text{mod} 53) = 23$$

$$51^1 (\text{mod} 53) = -2$$

$$51^2 (\text{mod} 53) = 51^1 \times 51^1 (\text{mod} 53) = -2 \times -2 = 4$$

$$51^4 (\text{mod} 53) = 51^2 \times 51^2 (\text{mod} 53) = 4 \times 4 = 16$$

$$51^8 (\text{mod} 53) = 51^4 \times 51^4 (\text{mod} 53) = 16 \times 16 (\text{mod} 53) = 44$$

$$51^{13} (\text{mod} 53) = 51^8 \times 51^4 \times 51^1 (\text{mod} 53) = 44 \times 16 \times -2 (\text{mod} 53) = 23$$

$$\text{Since}, b_0 \neq -1 \, or \, +1$$

$$b_1 = b_0^2 (\text{mod} 53)$$

$$= 23^2 (\text{mod} 53) = 529 (\text{mod} 53)$$

$$= 52 (\text{mod} 53)$$

$$= -1 \qquad \text{Since, } b_1 = -1, n = 53 \text{ is a prime number.}$$

Fermat's Little Theorem

Statement:

If p is a prime number and a is an integer not divisible by p, then:

$$a^{p-1} \equiv 1 \pmod{p}$$

or equivalently

$$a^{p} \equiv a \pmod{p}$$

Example 1: $p = 5$, $a = 2$

$2^4 = 16 \equiv 1 \pmod{5}$

Example 2: $p = 7$, $a = 3$

$3^6 = 729 \equiv 1 \pmod{7}$

Example 3: $p = 11$, $a = 2$

$2^{10} = 1024 \equiv 1 \pmod{11}$

Example 4: $p = 13$, $a = 5$

$5^{12} = 244140625 \equiv 1 \pmod{13}$

Example 5: $p = 17$, $a = 4$

$4^{16} = 4294967296 \equiv 1 \pmod{17}$

Primality Test

1. Pick a number n you want to test (must be greater than 3).
2. Choose a random integer a such that $1 < a < n-1$
3. Check the condition:

$$a^{n}-1 \equiv 1 (\text{mod } n)$$

If it fails $\rightarrow$ n is definitely composite.

If it holds $\rightarrow$ n is probably prime.

4. Repeat the test multiple times with different random values of a. The more iterations, the higher the confidence.

Time Complexity: $O(t.\ \log^3 n)$, same as Miller-Rabin test

Example: $n = 221$

Pick $a = 38$:

$38^{220} \equiv 1 (\text{mod } 221) \rightarrow$ suggests probably prime.

Pick $a = 24$:

$24^{220} \equiv 81 \not\equiv 1 (\text{mod } 221) \rightarrow$ shows definitely composite.

Thus, **221 is composite**.

Finding Modular Inverse

From Fermat's Little Theorem,

$a^{p-1} \equiv 1 \pmod{p}$, for prime p and a not divisible by p.

From this, we can rearrange,

$$a^{p-2} \equiv a^{-1} \pmod{p}$$

So the modular inverse of a modulo prime p is:

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Example:

Find the modular inverse of 3 mod 7

Solution:

$a = 3$

$p = 7$

$a^{-1} = 3^{-1} \equiv a^{7-2} \equiv 3^5 \equiv 243 \pmod{7} \equiv 5 \pmod{7}$

Therefore, the inverse of 3 mod 7 is 5 mod 7

Pseudocode for Modular Inverse using Fermat's Theorem

```
MODINV-FERMAT(a, m)
1 if gcd(a, m) ≠ 1
2 error "Inverse does not exist"
3 return MODULAR-EXPONENTIATION (a, m - 2, m)
```

Thank You!!!